Daily Quant Digest: 2026-10-27

Foundations Restart, day 2. Yesterday we laid the first brick in each track. Today's bricks sit directly on top of them.

Today's topics

Track	Topic	Difficulty
Probability & Statistics	FR Lesson 2: the language of events ("or", "and", "not"), the addition rule, inclusion-exclusion	Easy
Derivatives	FR Lesson 2: present value and discount factors (money today vs money later)	Easy
Algorithms & DS	FR Lesson 2: C++ refresher (`vector`, `string`, references, functions, loops); reverse the words in a string	Easy
Music	Queen, "Bohemian Rhapsody" (1975)	

---

1. Probability & Statistics: "Or", "And", "Not", and Inclusion-Exclusion

> **Where we are.** Lesson 1 gave us the room of possible worlds: each equally likely outcome is a person in the room, an event is a group of people, and $P(A) = |A|/|\Omega|$ (people in the group divided by people in the room). We also met $P(\text{not } A) = 1 - P(A)$. Today we learn to combine events with the words "or", "and", "not", and how to count a combined group without counting anyone twice.

1.1 The central picture: merging guest lists

You are organising a desk party. Two colleagues each hand you a guest list. Some names are on both lists. You want to know how many invitations to print.

The lazy answer is “length of list 1 plus length of list 2”. It is wrong, because the people on both lists get counted twice. The fix is obvious once you see it: **add the two lists, then subtract the people you counted twice.** That one sentence is the whole lesson. Everything else is that sentence written in symbols, and then the same sentence for three lists.

In the picture	In the maths	Symbol	Read aloud
A guest list	An event (a group of outcomes)	A, B	“A”, “B”
Everyone on list 1 **or** list 2 (or both)	Union	$A \cup B$	“A or B”
People on **both** lists	Intersection	$A \cap B$	“A and B”
People **not** on list A	Complement	A^c	“not A”
Two lists with no name in common	Disjoint (mutually exclusive) events	$A \cap B = \{\}$	“A and B cannot both happen”

Two small warnings about the words. In maths, “or” always means “one or the other **or both**” (the inclusive or). And $\cup$ looks like a cup that holds everything, while $\cap$ looks like a narrow arch that only covers the overlap. That is a decent way to remember which is which.

1.2 Counting with 1,000 traders

A bank has 1,000 traders. 400 trade FX. 300 trade rates. 100 of them trade both (they sit on the FX-rates hybrid desk). Pick one trader at random.

Draw it as two overlapping circles (a Venn diagram):

Region	How many	Why
FX and rates (the overlap)	100	given
FX only	$400 - 100 = 300$	FX traders who are not in the overlap
Rates only	$300 - 100 = 200$	rates traders who are not in the overlap
Neither	$1{,}000 - 300 - 200 - 100 = 400$	everyone else

The four regions add up to $300 + 200 + 100 + 400 = 1{,}000$. Good: every person is in exactly one region.

Now: how many trade FX **or** rates? Count the regions: $300 + 200 + 100 = 600$. The lazy count $400 + 300 = 700$ is 100 too many, because the 100 hybrid traders sat on both lists. So:

$$600 = 400 + 300 - 100.$$

1.3 The addition rule

Divide every count by 1,000 (the size of the room) and the counting sentence becomes the **addition rule**:

$$P(A \cup B) = P(A) + P(B) - P(A \cap B).$$

Symbols: A = "trades FX", B = "trades rates", $A \cup B$ = "trades at least one of them", $A \cap B$ = "trades both".
Read aloud: "the chance of A or B is the chance of A, plus the chance of B, minus the chance of both."
With numbers: $0.4 + 0.3 - 0.1 = 0.6$.

Special case: disjoint events. If no outcome is in both, the overlap is empty, $P(A \cap B) = 0$, and the rule becomes simply $P(A \cup B) = P(A) + P(B)$. On one die, "shows 1" and "shows 6" cannot both happen, so $P(1 \text{ or } 6) = 1/6 + 1/6 = 1/3$. Adding is only safe when you are sure there is no overlap.

"Not", and the escape hatch. The people **not** in $A \cup B$ are the ones on neither list: $P(\text{neither}) = 1 - 0.6 = 0.4$, matching the 400 in the table. Notice that "not (A or B)" is the same group as "(not A) and (not B)": a trader on neither list is not on list 1 and is also not on list 2. In symbols, $(A \cup B)^c = A^c \cap B^c$. This is called **De Morgan's law**, and it is why "at least one" questions are often easiest done through the complement, as we did yesterday with "at least one 6".

1.4 A small check on two dice

Roll a red and a blue die (36 equally likely cells). Let A = "at least one die shows 6" and B = "the total is 7".

- $|A| = 11$: the 6th row plus the 6th column, and the cell $(6,6)$ is in both, so $6 + 6 - 1 = 11$. (The addition rule is already hiding here.)
- $|B| = 6$: the diagonal from yesterday's grid.
- $|A \cap B| = 2$: the cells $(1,6)$ and $(6,1)$.

$$P(A \cup B) = \frac{11}{36} + \frac{6}{36} - \frac{2}{36} = \frac{15}{36} = \frac{5}{12}.$$

I checked by listing all 36 cells by computer: 15 of them qualify.

1.5 Three lists: inclusion-exclusion

Now three guest lists A, B, C. Let's do the same "add, then fix the double counts" and watch carefully what happens to a person on **all three** lists.

1. **Add the three lists:** $|A| + |B| + |C|$. Someone on all three lists is counted **3** times.

2. **Subtract the three pairwise overlaps:** $|A \cap B|$, $|A \cap C|$, $|B \cap C|$. Our triple person is in all three overlaps, so we remove them 3 times. Their count is now $3 - 3 = 0$. They have vanished!
3. **Add back the triple overlap** $|A \cap B \cap C|$ once. Now they are counted $0 + 1 = 1$ time. Correct.

Check the others: a person on exactly one list is counted once in step 1 and never touched again. A person on exactly two lists is counted 2 times in step 1, minus 1 in step 2 (only one pairwise overlap contains them), total 1. Everyone ends up counted exactly once. So:

$$\begin{aligned} P(A \cup B \cup C) = P(A) + P(B) + P(C) \\ - P(A \cap B) - P(A \cap C) - P(B \cap C) \\ + P(A \cap B \cap C). \end{aligned}$$

Read aloud: “add the singles, subtract the pairs, add back the triple.” The pattern of plus, minus, plus continues for four or more events (add singles, subtract pairs, add triples, subtract quadruples, …). That alternating pattern is called **inclusion-exclusion**.

Where the analogy stops working. Guest lists are finite, and we can count heads. Later (Unit D) events will be regions of a line or a square, with infinitely many outcomes, and we will measure them by length or area instead of counting. The addition rule still holds word for word: area of a union is the areas added, minus the overlap.

1.6 Common mistakes

- **Adding overlapping probabilities.** “Each roll has a 1/6 chance of a 6, so in 6 rolls the chance of at least one 6 is $6 \times 1/6 = 1$.” Absurd: you could easily roll no 6. The six events overlap (you can roll a 6 on roll 1 **and** roll 4), so you cannot just add them. Through the complement, the true answer is $1 - (5/6)^6 \approx 0.665$. (Multiplying the $5/6$ s uses independence, Lesson 7; for now, trust it or check it by listing.)
- **Forgetting the triple term.** With three events, people stop after “subtract the pairs”. Do the bookkeeping from 1.5 on a person in all three groups and you will never forget it again.
- **Confusing disjoint with “unrelated”.** Disjoint means the events **cannot happen together**. That is a very strong link between them, not an absence of one. (Lesson 7 makes this precise.)

1.7 Self-check

1. One card from a standard 52-card deck. What is the probability it is a heart or a king?
2. Of 1,000 traders, 400 trade FX, 300 rates, 100 both (the table above). What is the probability a random trader trades exactly one of the two?
3. Three dice. Use inclusion-exclusion to count the outcomes with at least one 6 out of 216, then check with the complement.

1.8 Interview question: the divisibility sieve

(A classic warm-up at trading firms. It tests whether you see the overlap.)

Pick an integer uniformly at random from 1 to 1,000.

a. What is the probability it is divisible by 6 or by 10?

b. What is the probability it is divisible by 2, by 3, or by 5?

c. What is the probability it is divisible by none of 2, 3 and 5?

1.9 Solution

A useful counting fact first. How many numbers from 1 to 1,000 are divisible by 6? They are $6, 12, 18, \ldots$, one every 6 steps, and the last one is $6 \times 166 = 996$. So there are 166. In general the count is $\lfloor 1000/k \rfloor$: divide and throw away the remainder (the symbol $\lfloor \cdot \rfloor$ means "round down").

Second fact: what "divisible by both" means. Divisible by 6 **and** by 10 means divisible by their lowest common multiple, 30 (not $6 \times 10 = 60$: 30 itself works). Divisible by 2 and 3 means divisible by 6, and so on.

(a) A = "divisible by 6": 166 numbers. B = "divisible by 10": 100. $A \cap B$ = "divisible by 30": $\lfloor 1000/30 \rfloor = 33$.

$$|A \cup B| = 166 + 100 - 33 = 233, \qquad P = \frac{233}{1000} = 0.233.$$

(b) Singles: by 2: 500, by 3: 333, by 5: 200. Pairs: by 6: 166, by 10: 100, by 15: 66. Triple: by 30: 33.

$$500 + 333 + 200 - 166 - 100 - 66 + 33 = 734, \qquad P = 0.734.$$

(c) This is "not (A or B or C)", the complement: $1{,}000 - 734 = 266$, so $P = 0.266$.

Interview trick. For part (c), think of a stretch of 30 consecutive numbers. Half are odd; of those, two thirds are not multiples of 3; of those, four fifths are not multiples of 5. So a fraction $\frac{1}{2} \times \frac{2}{3} \times \frac{4}{5} = \frac{8}{30} \approx 0.2667$ survive. Since 1,000 is not a multiple of 30, the exact answer differs slightly: 0.266 versus 0.2667. Saying "about 26.7%, exactly 266 out of 1,000 after the edge effect" sounds very good in an interview.

Sanity check. I counted all 1,000 numbers by computer: 233 for (a), 734 for (b), 266 for (c). Also, (b) must be at least 0.5 (the evens alone) and at most $0.5 + 0.333 + 0.2 = 1.033$ (the lazy sum, which overcounts). 0.734 sits sensibly in between.

Self-check answers. (1) 13 hearts + 4 kings, minus the king of hearts counted twice: $16/52 = 4/13$. (2) FX only (300) plus rates only (200): $500/1000 = 0.5$. Or: $P(A \cup B) - P(A \cap B) = 0.6 - 0.1$. (3) Singles: each die is a 6 in 36 outcomes, total $3 \times 36 = 108$.

Pairs: two given dice both 6 in 6 outcomes, $3 \times 6 = 18$. Triple: $(6, 6, 6)$, 1. So $108 - 18 + 1 = 91$. Complement: $216 - 5^3 = 216 - 125 = 91$. They match.

---

2. Derivatives: Present Value and Discount Factors

> **Where we are.** Lesson 1 taught how money grows when it is lent: simple interest $P(1 + rt)$ for short deposits (with ACT/360 day counts in USD), and continuous compounding Pe^{rt}. Today we run the snowball **backwards**: if I will receive money later, what is it worth today?

2.1 The scene

A Hong Kong electronics exporter has shipped goods to a US retailer. The invoice says: **USD 10 million, payable in 182 days.** The exporter needs cash now to pay its own suppliers. Its bank offers to buy the invoice and pay **USD 9.79 million today** (this is called invoice discounting). Is that a fair price?

Who is on each side? The exporter wants cash now and is willing to give up a bit of value for it. The bank is happy to wait 182 days because it is paid for waiting. Both sides need the same tool to judge the deal: the **present value** of money that arrives later.

2.2 The central picture: future money is a foreign currency

You work in FX, so here is the picture that makes this common sense. **Treat "dollars in 182 days" as a different currency from "dollars today".** You cannot add USD and JPY without first converting one into the other at an exchange rate. In exactly the same way, you cannot add a dollar today and a dollar next year without converting. The exchange rate between them is the **discount factor**.

In the picture	In the maths
A foreign currency: "dollars delivered at time t"	A cash flow at date t
The home currency: "dollars today"	Present value (PV)
The exchange rate from future dollars into today dollars	The **discount factor** $DF(t)$
Converting an amount into home currency	$\text{PV} = \text{cash flow} \times DF(t)$
The rate sheet showing a rate for every date	The **discount curve**

The key sentence: $DF(t)$ **is the price today of receiving exactly 1 dollar at time** t**.** If $DF(1) = 0.96$, then a promise of 1 dollar in one year is worth 96 cents now.

2.3 Where the discount factor comes from

Run Lesson 1's snowball backwards. If you put x dollars on deposit today at simple rate r for time t, it grows to $x(1+rt)$. Which x grows to exactly 1 dollar? Set $x(1+rt)=1$ and divide:

$$DF(t)=\frac{1}{1+rt}\quad\text{(simple interest)},\qquad DF(t)=\frac{1}{e^{rt}}=e^{-rt}\quad\text{(continuous)}.$$

Read aloud: "the discount factor is one divided by the growth factor." Whatever 1 dollar today grows into, you divide by that to come back.
Why it must be below 1 (when rates are positive): a dollar today can be deposited and become *more* than a dollar later. So a dollar later is worth less than a dollar today. Anyone who disagreed would sell you future dollars at par and you would take the free money.

With numbers, for our exporter. The USD deposit rate for 182 days is 4.00%, ACT/360:

$$t=\frac{182}{360}=0.50556,\qquad 1+rt=1+0.04\times 0.50556=1.020222,$$

$$DF=\frac{1}{1.020222}=0.980179,\qquad \text{PV}=10{,}000{,}000\times 0.980179=9{,}801{,}786\ \text{USD}.$$

So the "fair" price at the deposit rate is about 9.80 million. The bank offers 9.79 million, about 11,800 USD less. Is the bank being greedy? Not necessarily: the bank also takes the risk that the US retailer never pays, and charges for that risk. The 9.80 million is the value **if the payment is certain**. Real credit risk lowers the price (a later lesson). The skill today is knowing the risk-free benchmark so you can see how much you are being charged for everything else.

Notice how close the continuous version is: $e^{-0.04\times 0.5}=0.980199$ (taking exactly half a year). The conventions differ in the 5th decimal over 6 months, but as Lesson 1 showed, the gap grows with time. On a real trade you use the convention the market quotes.

2.4 The superpower: adding cash flows at different dates

Most deals pay several amounts at several dates. Once every cash flow is converted into "today dollars", they are all the same currency and you can simply add them.

Worked example. The exporter's owner is selling a small factory. Two offers:

- **Offer A:** USD 9.60 million today.
- **Offer B:** USD 3.0 million in 6 months, 3.5 million in 1 year, 3.5 million in 2 years. That is **10.0 million** in total.

B "pays more", 10.0 versus 9.6. But that 10.0 adds dollars of three different currencies. Today's USD curve (continuously compounded zero rates) and its discount factors $DF=e^{-rt}$:

Time t (years)	Rate r	$DF(t) = e^{-rt}$	Cash flow (USD)	PV = cash flow $\times$ DF
0.5	4.00%	0.980199	3,000,000	2,940,596
1	3.90%	0.961751	3,500,000	3,366,127
2	3.70%	0.928672	3,500,000	3,250,351
			Total	**9,557,074**

Check one row by hand: $e^{-0.039 \times 1} = e^{-0.039}$. Lesson 1 told us $e^x \approx 1 + x$ for small x, so this is about $1 - 0.039 = 0.961$; the exact value is 0.961751. Then $3{,}500{,}000 \times 0.961751 = 3{,}366{,}127$.

Offer B is worth **9.56 million** today, so Offer A (9.60 million cash) is better by about **43,000 USD**, even though its headline number is 400,000 smaller. That is the whole point of present value.

What if rates rise? Suppose every rate on the curve goes up by 1 percentage point before the owner signs. Recomputing with 5.00%, 4.90%, 4.70% gives a PV for Offer B of **9,444,553**. Higher rates mean smaller discount factors, so promises of future money are worth less. This "rates up, value of future cash down" link is exactly why bond prices fall when yields rise (Lesson 8).

2.5 When the discount factor went above 1

Under negative interest rates the snowball melts, and the formula says $DF = e^{-rt}$ with $r < 0$, which is **bigger than 1**. A dollar (or franc, or euro) in the future was worth *more* than one today. This was not a theory: the Swiss National Bank held its policy rate at minus 0.75% from 2015 to 2022, and in August 2019 the 10-year German government bond yield fell to around minus 0.7%. Investors were paying the German government to hold their money, because storing large sums of cash safely is expensive and they wanted the safest asset available. At minus 0.75% for one year, $DF = e^{0.0075} \approx 1.0075$: you would pay about 1.0075 francs today to receive 1 franc next year. Desks whose systems quietly assumed "a discount factor is always below 1" had bugs to fix.

Where the analogy stops working. A real foreign currency can be held and spent anywhere. "Future dollars" cannot be spent until the date arrives, and they only exist as promises, so who makes the promise matters (credit risk). The exchange-rate picture is perfect for the arithmetic, but remember that a promise from a weak borrower is a weaker "currency".

2.6 Likely interview questions

1. **What is a discount factor?** The price today of receiving 1 unit of currency at a future date, risk-free. It is 1 over the growth factor: $1/(1 + rt)$ with simple interest, e^{-rt} with continuous compounding.

2. **Which is worth more today: 100 in 1 year with $DF(1) = 0.96$, or 104 in 2 years with $DF(2) = 0.92$?** $100 \times 0.96 = 96.00$ versus $104 \times 0.92 = 95.68$. The 100 in 1 year, by 32 cents per 100.
3. **What is the PV of USD 1 million received in 3 years at 5%?** With annual compounding: $1{,}000{,}000 / 1.05^3 = 863{,}838$. With continuous compounding: $1{,}000{,}000\, e^{-0.15} = 860{,}708$. Always ask which convention.

---

3. Algorithms & Data Structures: Reverse the Words in a String

> **Where we are.** Lesson 1 gave us Big-O: count the steps, keep the fastest-growing term, and remember that $O(n^2)$ is fine for a few thousand items but hopeless for 10^5. Today is a C++ refresher: `std::vector`, `std::string`, references, functions and loops. We use a problem simple enough that all our attention can go on the language, and still find a hidden $O(n^2)$ trap.

3.1 The central picture: a train of carriages

Picture a string as a **train**: a row of carriages, each holding one character, numbered from 0. A `std::vector` is the same kind of train, but the carriages can hold anything (numbers, or even whole strings).

In the picture	In C++
A train of carriages, each holding one character	`std::string s`
A train whose carriages hold any type `T`	`std::vector<T>`
Carriage number `i`	`s[i]` (counting from 0)
Adding a carriage at the back: cheap	`s.push_back(c)`, `s += "..."`
Adding a carriage at the front: every carriage must shuffle back	building `word + " " + s`, or `s.insert(0, ...)`
Giving someone the train itself, not a photocopy	a **reference**, `std::string&`

Remember the asymmetry in rows 4 and 5. Adding at the back is fast. Adding at the front means moving (or copying) every carriage already there. That single fact is today's whole complexity lesson.

3.2 The problem

Given a string `s` of words separated by one or more spaces, possibly with spaces at the start and end, return the words in **reverse order**, separated by **exactly one** space, with no leading or trailing spaces. A word is any run of non-space characters.

Constraints: $0 \le n \le 2 \times 10^5$ characters.

Examples:

- `"buy EURUSD at 1.16"` gives `"1.16 at EURUSD buy"`.
- `"  sell  500  USDJPY  "` gives `"USDJPY 500 sell"` (extra spaces vanish).
- `"hedge"` gives `"hedge"`.
- `"  "` gives `""` (no words at all).

3.3 Brute force first: glue each word onto the front

The most natural method: read the words left to right, and put each new word **in front of** the answer so far.

Word read	Answer after
`buy`	`buy`
`EURUSD`	`EURUSD buy`
`at`	`at EURUSD buy`
`1.16`	`1.16 at EURUSD buy`

Correct. But look at the cost. The line `answer = word + " " + answer;` builds a brand-new string and copies **every character already in the answer** into it. If there are w words of length about L, the copies cost about $L, 2L, 3L, \ldots, wL$. As we added up in Lesson 1, $1 + 2 + \cdots + w = w(w+1)/2$, so the total is about $Lw^2/2$. With n characters, w is about n/L, giving about $n^2/(2L)$ steps: $O(n^2)$. It is the "adding carriages at the front" trap.

3.4 Better: collect the words, then read them backwards

Insight: the slow part was building the answer from the front. So **separate the two jobs**. First scan left to right and put each word into a `std::vector<std::string>` (adding at the back, cheap). Then walk the vector from the last word to the first, appending each to the answer (again adding at the back, cheap). Every character is copied a constant number of times: $O(n)$ time. Extra space: the vector holds a copy of all the words, $O(n)$.

3.5 Best: two reversals, in place

Can we avoid the extra copy? Here is a lovely trick. Reverse the **whole** string, character by character:

`"buy EURUSD at 1.16"` becomes `"61.1 ta DSRUE yub"`.

The **words** are now in the right order (1.16, at, EURUSD, buy), but each word is spelled backwards. So reverse **each word** on its own:

`"61.1 ta DSRUE yub"` becomes `"1.16 at EURUSD buy"`.

Done. Reversing twice cancels out inside each word, but the outer reversal moved the words. While we fix each word, we also squeeze out extra spaces using two positions: `read` (where we are looking) and `write` (where the next kept character goes). `write` never runs ahead of `read`, so we never overwrite a character we still need. Every character is touched a constant number of times: $O(n)$ time, and $O(1)$ extra space (a few integers).

The program times all three on a string of 200,000 characters. On the test machine: front-gluing took about 180 ms, split-and-join about 5 ms, in-place under 1 ms. The $O(n^2)$ version is roughly 35 times slower than the $O(n)$ one already, and doubling n would make the gap twice as wide.

3.6 C++ solution

```
#include <algorithm>
#include <cassert>
#include <chrono>
#include <iostream>
#include <random>
#include <string>
#include <vector>

// Version 1: glue each word onto the FRONT. O(n^2).
std::string reverseWordsPrepend(const std::string& s) {
    std::string answer;
    const int n = static_cast<int>(s.size());
    int i = 0;
    while (i < n) {
        while (i < n && s[i] == ' ') ++i;          // skip spaces
        if (i == n) break;
        int j = i;
        while (j < n && s[j] != ' ') ++j;          // s[i..j-1] is one word
        std::string word = s.substr(i, j - i);
        if (answer.empty()) answer = word;
        else answer = word + " " + answer;         // copies all of answer
        i = j;
    }
    return answer;
}

// Version 2: split into a vector of words, then join them from the back. O(n).
std::vector<std::string> splitWords(const std::string& s) {
    std::vector<std::string> words;
    std::string current;
    for (char c : s) {                             // range-for over characters
        if (c == ' ') {
            if (!current.empty()) {
                words.push_back(current);
                current.clear();
            }
        } else {
            current.push_back(c);
        }
    }
    if (!current.empty()) words.push_back(current); // last word has no space after
    return words;
}

std::string reverseWordsSplit(const std::string& s) {
    std::vector<std::string> words = splitWords(s);
    std::string answer;
```

```
    for (int k = static_cast<int>(words.size()) - 1; k >= 0; --k) {
        answer += words[k];                          // += appends, no full copy
        if (k > 0) answer += ' ';
    }
    return answer;
}

// Version 3: work inside the caller's own string. O(n) time, O(1) extra space.
void reverseWordsInPlace(std::string& s) {
    std::reverse(s.begin(), s.end());                // "ab cd" -> "dc ba"
    const int n = static_cast<int>(s.size());
    int write = 0;                                   // next free slot in the answer
    int read = 0;
    while (read < n) {
        if (s[read] == ' ') { ++read; continue; }    // skip spaces
        if (write > 0) s[write++] = ' ';             // one space between words
        int start = write;                           // where this word will begin
        while (read < n && s[read] != ' ') s[write++] = s[read++];
        std::reverse(s.begin() + start, s.begin() + write); // fix the letters
    }
    s.resize(write);                                 // chop off leftover characters
}

int main() {
    struct Case { std::string input; std::string expected; };
    const std::vector<Case> cases = {
        {"buy EURUSD at 1.16", "1.16 at EURUSD buy"},
        {"   sell   500   USDJPY  ", "USDJPY 500 sell"},
        {"hedge", "hedge"},
        {"     ", ""},
        {"a b  c", "c b a"},
    };
    for (const auto& c : cases) {
        std::string a = reverseWordsPrepend(c.input);
        std::string b = reverseWordsSplit(c.input);
        std::string d = c.input;                     // copy, because version 3 edits it
        reverseWordsInPlace(d);
        std::cout << "[" << c.input << "] -> [" << d << "]\n";
        assert(a == c.expected && b == c.expected && d == c.expected);
    }
    // (The full .cpp also runs 50,000 random cross-checks and the timing test.)
    return 0;
}
```

The full file is `2026-10-27.cpp`. Compile with `g++ -std=c++17 -O2 -Wall 2026-10-27.cpp`. It passes all five examples, 50,000 random strings on which all three versions agree, and prints the timings above.

3.7 Walkthrough of the C++

`std::string` (from `<string>`) is a vector of `char` with extras. `s.size()` is its length; `s[i]` is carriage `i`; `s.empty()` asks "length zero?"; `s.clear()` empties it; `s.push_back(c)` appends one character; `s += "text"` or `s += 'c'` appends at the back. `s.substr(i, len)` returns a **new** string copied from position `i`, of length `len`. `s.resize(k)` keeps only the first `k` characters.

`std::vector<std::string>` is a train whose carriages are whole strings. `words.push_back(current)` copies `current` onto the back. `words[k]` is word number `k`, and `words.size()` is how many there are.

`'c'` versus `"c"`. Single quotes make one `char`; double quotes make a string literal. We compare `s[i] == ' '` with single quotes because `s[i]` is a single character.

`const std::string& s` in versions 1 and 2: a **reference** (no copy of a 200,000-character string), and `const` (the function promises not to change it). This is the default way to pass anything bigger than a number into a function.

`std::string& s` in version 3, *without* const: the function receives the caller's own string and edits it. That is why `main` first makes a copy, `std::string d = c.input;`, and passes `d`. The return type is `void`: nothing is returned, because the result is already sitting in the caller's string.

Returning a `std::string` or `std::vector` by value (versions 1 and 2) looks expensive, but it is not: modern C++ **moves** the result out of the function (it hands over the train instead of copying every carriage). So "return a big object by value" is normal, fast, modern style.

Three kinds of loop. `while (i < n && s[i] == ' ') ++i;` is a `while` loop with a compound condition. The order inside `&&` matters: C++ checks `i < n` first and **stops** if it is false, so `s[i]` is never read past the end. This is called short-circuit evaluation. `for (int k = size - 1; k >= 0; --k)` counts down. We cast `words.size()` to `int` first: `size()` is unsigned, and an unsigned `0 - 1` wraps around to a huge number, so `k >= 0` would always be true. `for (char c : s)` is the range-for from Lesson 1, here over characters.

`s[write++] = s[read++];` The `++` *after* the name means "use the current value, then add 1". So this line copies carriage `read` into carriage `write`, then moves both positions forward by one. (`++i` *before* the name adds first, then uses the value.)

Iterators and `std::reverse`. `s.begin()` is an **iterator**: a bookmark pointing at the first character. `s.end()` points one step **past** the last character. `std::reverse(first, last)` (from `<algorithm>`) reverses everything from `first` up to, but not including, `last`. `s.begin() + start` moves the bookmark forward `start` steps, so `std::reverse(s.begin() + start, s.begin() + write)` reverses exactly the word we just copied. The "up to but not including" (half-open range) convention is used everywhere in the standard library.

The test harness. As yesterday, `assert` checks every version against the expected answers, then the full file compares all three versions on 50,000 random strings made only of spaces and a few letters (lots of awkward multiple-space cases). The timing part uses a **lambda**, `[&] { r1 = reverseWordsPrepend(big); }`: a small unnamed function written inline. The `[&]` means "this function may use the surrounding variables by reference". Lambdas get a full lesson later; for now read it as "a block of code I can pass around".

---

4. Music: Queen, “Bohemian Rhapsody” (1975)

Context

“Bohemian Rhapsody” is on *A Night at the Opera* (1975), Queen’s fourth album. It was written by Freddie Mercury and produced by Roy Thomas Baker with the band. At almost six minutes, with no chorus and several completely different sections, it broke every rule of a pop single.

The story behind it

Mercury had been carrying pieces of the song around for years, and he arrived at the sessions with it largely worked out, guiding the others through it section by section. The recording was spread over several studios in the summer of 1975, starting at Rockfield in Wales. The famous operatic middle section was built by Mercury, Brian May and Roger Taylor singing their parts again and again and stacking the takes, on analogue tape that was bounced down so many times it wore thin.

The record company and others thought six minutes was far too long for radio. Mercury and the band refused to cut it. The London DJ Kenny Everett, a friend of Mercury’s, got hold of a copy and played it repeatedly on Capital Radio before release, and demand followed. It went to number one in the UK for nine weeks over Christmas 1975. Its promotional film, directed by Bruce Gowers, with the four faces lit from above in the dark, became a model for the music video era. After Mercury’s death in 1991 it went to number one in the UK again, and the head-banging scene in the 1992 film *Wayne’s World* took it back up the US charts.

Why people love it

It is a whole mini-opera in six minutes: a ballad, a guitar solo, a mock opera, a hard rock outburst, and a quiet ending. It should not work, and it does, because each section is a strong piece of music on its own and the joins are dramatic. It is also pure fun to sing along to, which is why it fills stadiums and cars alike.

What to listen for

- **The opening.** Close vocal harmony (Mercury’s voice stacked on itself) with almost no accompaniment, before the piano takes over.
- **Brian May’s guitar solo.** It is a melody you could sing, played on the guitar he built with his father, and it is the bridge from the ballad into the opera.
- **The opera section.** Listen in headphones to the voices jumping between left and right, and to the sheer thickness of the stacked choir.
- **The switch to hard rock.** The sudden burst of guitar and drums after the choir is one of the great gear changes in rock.
- **The ending.** It winds down to piano and a soft gong, the opposite of a big finish.